
MEETING SUMMARIZER

Software Specification Requirements for

MAJOR PROJECT I

by

Name	Enrollment Number
Rishika Raj	R2142220279
Anshi Mehta	R2142220034
Dhananjay Baliyan	R2142220915

Under the guidance of

Dr.Pavinder

Assistant Professor

Data Science Cluster(SoCS)



SCHOOL OF COMPUTER SCIENCE
UNIVERSITY OF PETROLEUM AND ENERGY STUDIES
Dehradun-248007

CANDIDATE'S DECLARATION

I/We certify that the project work titled "AI-Powered Meeting Summarizer," which was submitted to the Department of Systemics, School of Computer Science, University of Petroleum & Energy Studies, Dehradun, in partial fulfilment of the requirements for the award of the Degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING with specialization in DevOps and Data Science, is an authentic record of my/our own work completed between August 2025 to December 2025 under the guidance of Dr. Pavinder.

(Rishika Raj, Anshi Mehta, Dhanajay Baliyan)

Date: 8 December, 2025

Dr. Pavinder
Project Guide

ACKNOWLEDGEMENT

We wish to express our deep sense of gratitude to our project guide, Dr. Pavinder, for her valuable guidance, constant encouragement, and unwavering support throughout the development of our project titled “AI-Powered Meeting Summarizer.” This project would not have been possible without her insightful suggestions and constructive feedback at every stage of the work.

We would like to sincerely thank our respected Dr. Avita Katal and Dr. Hitesh Kumar Sharma, *Cluster Head – Cloud Computing and Virtualization, School of Computer Science*, for their continuous support and motivation, which greatly contributed to the successful completion of this project.

We are also grateful to the Dean, School of Computer Science, UPES, for providing us with the necessary infrastructure and facilities required to carry out our project work smoothly. We extend our sincere thanks to our Course Coordinators, Dr. Vinod Kumar and Dr. Kumar Gaurav, and our Activity Coordinators, Dr. Justin Joseph and Dr. Kumar Gaurav, for their timely guidance, support, and valuable information during the course of the project.

We would like to thank our friends and classmates for their cooperation, encouragement, and constructive criticism, which helped us improve our work significantly. Finally, we express our heartfelt gratitude to our parents for their unconditional love, encouragement, and constant moral support, without which this achievement would not have been possible.

Name	Roll No.
Rishika Raj	R2142220279
Anshi Mehta	R2142220034
Dhananjay Baliyan	R2142220915

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Target Beneficiaries	1
1.3	Project Scope	2
1.3.1	Core Functionalities	2
1.3.2	System Capabilities	2
2	Project Description	2
2.1	Reference Algorithm	2
2.2	Data Formats & Structures	3
2.3	SWOT Analysis	3
2.4	Project Features	3
2.5	User Classes and Characteristics	4
2.6	Design and Implementation Constraints	4
2.7	Design Diagrams	5
2.8	Assumptions and Dependencies	6
3	System Requirements	7
3.1	User Interface	7
3.2	Software Interface	7
3.3	Database Interface	8
4	Non-Functional Requirements	8
4.1	Performance Requirements	8
4.2	Security Requirements	9
4.3	Software Quality Attributes	9
4.4	Other Requirements	9
5	Project Plan	10

List of Figures

1	Use Case Diagram	5
2	Data Flow Diagram	5
3	Grammar Pipeline	6
4	Multimodal RAG Pipeline	6
5	Sentiment Analysis Pipeline	6

List of Tables

1	SWOT Analysis	3
2	User Classes and Characteristics	4

1 Introduction

Meetings produce a lot of unstructured information in the fast-paced digital world of today, which is sometimes forgotten, misunderstood, or never evaluated. In order to overcome this difficulty, our project presents a Meeting Summarizer, a completely local, privacy-preserving tool that turns audio recordings of meetings into precise transcripts and insightful summaries. This tool ensures that important insights are recorded while significantly cutting down on the amount of time spent reviewing discussions.

In the previous works, Sadia et al. (2024) [1] presents a new benchmark dataset for Urdu meeting summarization and evaluates several deep learning models for generating abstractive summaries. It highlights the effectiveness of models like ur_mT5 and GPT-3.5, while addressing the challenges of working with a low-resource language. The study contributes a valuable resource and demonstrates promising results, despite limitations in generalizability and reliance on proprietary tools.

Bramachari(2024) [2] The paper proposes a language-neutral framework for meeting summarization by combining multiple NLP techniques and using the Longformer encoder-decoder model to handle long, multilingual discussions. It aims to improve accessibility and understanding in global teams. While the approach is inclusive and well-suited for lengthy meetings, the paper lacks detailed evaluation metrics or real-world validation.

Vadlamudi, G (2022) [3] The paper presents a meeting summarization system using TF-IDF and PageRank to generate concise summaries from Microsoft Teams transcripts. It aims to simplify long transcripts and highlight key points. While practical and easy to implement, it relies on traditional extractive methods and lacks advanced evaluation.

1.1 Purpose

The goal of the Meeting Summarizer project is to create a comprehensive, locally deployable system that uses cutting-edge voice recognition and natural language processing (NLP) technology to automatically transcribe and summarize meeting audio. This system combines MPEG for reliable audio preprocessing, Ollama's local Large Language Model (LLM) server for context-aware text summarization, and whisper.cpp, a high-performance C++ implementation of OpenAI's Whisper.

The solution, which was created to maximize organizational efficiency, guarantees the accurate, efficient, and privacy-preserving extraction of important insights from spoken dialogue. The system offers structured transcripts and actionable summaries by utilizing ascalable and flexible architecture, real-time streaming capabilities, and an intuitive Gradio interface. In business settings, this promotes enhanced documentation workflows, quicker decision-making, and sophisticated human-computer interaction.

1.2 Target Beneficiaries

A wide range of users and domains that depend on effective meeting documentation, real-time summarization, and information extraction gain from this initiative, including:

1. Enhancing meeting productivity, automating minutes of meetings (MoM), and optimizing knowledge management workflows are all priorities for corporate teams and enterprises.
2. Research groups and educational institutions: helping with seminar documentation, group research discussions, and lecture recording.
3. Supporting the transcribing of court cases, hearings, and regulatory meetings with an emphasis on privacy and traceability is the responsibility of the legal and compliance sectors.
4. Enabling voice-to-text recording of clinical conversations, doctor-patient exchanges, and medical briefings is part of healthcare and clinical administration.
5. Creating easily accessible meeting summaries for dispersed teams is one way to support asynchronous communication in remote and hybrid workforces.
6. Providing a modular, local-first platform enabling developers and open-source communities to create adaptable AI tools in the fields of natural language processing and speech recognition.

1.3 Project Scope

The project's main goal is to automate the transcription and summarizing of spoken material during meetings by combining cutting-edge voice recognition and natural language processing (NLP) techniques. It uses whisper to highlight a local-first architecture. FFmpeg for real-time audio preprocessing, Ollama for context-aware LLM-driven summarization, and cpp for on- device audio-to-text conversion.

The system supports several audio formats, is flexible, lightweight, and protects privacy. Gradio offers an easy-to-use user interface. Exact transcript creation, extractive and abstractive summarization, integration of user-defined models, and potential expansion toward multilingual support and streaming input capabilities are all included in the scope.

1.3.1 Core Functionalities

- **Automated Speech-to-Text Transcription:** Using whisper.cpp, conference audio may be accurately and CPU-efficiently converted into structured text, supporting a variety of languages and accents.
- **Context-aware summarization** is the process of using local LLMs through Ollama to produce logical, human-like summaries that include important choices, next steps, and points of discussion.
- Ensuring consistent transcription performance across different recording sources requires robust audio preprocessing, which involves integrating FFmpeg to standardize audio inputs (e.g., mono 16kHz WAV format).
- **Full control over sensitive meeting data** is made possible by privacy-preserving local deployment, which runs the complete pipeline on local hardware rather than relying on cloud APIs.
- With Gradio's user-friendly, browser-based interactive user interface, users may submit audio recordings, check transcripts, and obtain summaries with little technical hassle.

1.3.2 System Capabilities

- **Flexible integration and scalability** across various use cases are made possible by the architecture's support for modular components including audio preprocessing (FFmpeg), speech-to-text (Whisper), and summarization (LLMs via Ollama).
- **Local-First and Privacy-Preserving Execution:** This approach ensures data security, confidentiality, and regulatory compliance in sensitive contexts by doing all operations on local computers without the need for cloud-based APIs.
- **Real-Time Audio Processing and summary:** This feature is perfect for live meetings, seminars, and group projects since it facilitates real-time transcription and summary operations.
- **Cross-Platform Integration Capability:** Offers a backend-ready framework that can be included into research software, educational platforms, and business communication tools.

2 Project Description

2.1 Reference Algorithm

Abstractive Text Summarization is used by the Meeting Summarizer and is based on transformer-based models, notably T5 (Text-To-Text Transfer Transformer) or BART (Bidirectional and Auto-Regressive Transformers).

- Hugging Face Transformers were used to refine BART or T5 models.
- OpenAI Whisper or the Google Speech-to-Text API are used to translate audio inputs into text.

2.2 Data Formats & Structures

For transcription and summarization, the system transforms unstructured audio data into structured text outputs. Among the most important data formats and structures are:

- **Audio Files (MP3, WAV, M4A, FLAC):** For accuracy and compatibility, raw meeting recordings are preprocessed using FFmpeg into a standardized 16kHz mono WAV format after being uploaded in a variety of formats.
- **Text Transcripts (Plain Text /.txt):** Produced by whisper.cpp, these files provide the complete meeting transcription, along with timestamps and, if relevant, speaker conversations.
- **Ollama's LLM created the summarized text (JSON or plain text),** which condenses important decisions, action items, and organized summaries for convenient reading and archiving.
- **Contextual metadata (in JSON format):** To provide reproducibility and version control, it contains the user-provided meeting context, chosen models (such as Whisper and LLM), and processing settings.

2.3 SWOT Analysis

Strengths

1. **Automated Summarization:** Produces succinct and logical meeting summaries automatically by utilizing transformer-based models (such as BART, T5, and Pegasus) and cutting-edge natural language processing.
2. **Cloud integration:** Makes it possible to process meeting transcripts in a scalable and effective manner utilizing cloud platforms such as AWS, Azure, or GCP.
3. **Time-saving:** Significantly less manual note-taking is required, which saves teams time and boosts output.
4. **Multi-format Support:** Adaptable to a variety of meeting formats (e.g., Zoom, MS Teams, Google Meet, etc.), it can process input via audio, video, or text transcripts.

Weaknesses

1. **Speech-to-Text Errors:** The quality of the transcription determines the accuracy; summarization may be impacted by background noise, accents, or subpar audio.
2. **Model Complexity:** Training and fine-tuning large transformer models is costly and demands a large amount of computational power.
3. **Context Loss:** In lengthy meetings, abstractive models may misunderstand technical talks or fail to capture important conclusions.
4. **Data privacy issues:** Using cloud platforms to handle private meeting data increases the potential of security and compliance issues

Opportunities

1. **Enterprise Integration:** For automated documentation, it can be integrated with enterprise platforms such as Jira, Confluence, or Slack.
2. **Multilingual Support:** Entering the multilingual summarization market gives possibilities to varied teams and international marketplaces.
3. **Voice Assistant Integration:** Enables voice-activated summary creation when integrated into AI assistants such as Google Assistant or Alexa.
4. **Domain-specific meeting data,** such as legal, medical, and IT, can be fine-tuned to greatly increase the relevance and accuracy of summaries.

Threats

1. **Competition:** Due to the rapid advancement of generative AI, well-established firms like Microsoft Copilot, Fireflies.ai, and Otter.ai will be fierce rivals.
2. **Data Security & Compliance:** Meeting content storage raises legal and regulatory issues, particularly in industries like finance and healthcare.
3. **Dependency on External APIs:** Service interruptions may result from using third-party transcription or summarizing services (like Google Speech-to-Text).

User Resistance: Because of worries about accuracy or trust, traditional businesses may be reluctant to implement AI-driven summarization

2.4 Project Features

- Upload audio/text meeting input
- Real-time audio transcription
- Generate summarized meeting notes
- Download/export summary in PDF or TXT
- Search past meeting summaries

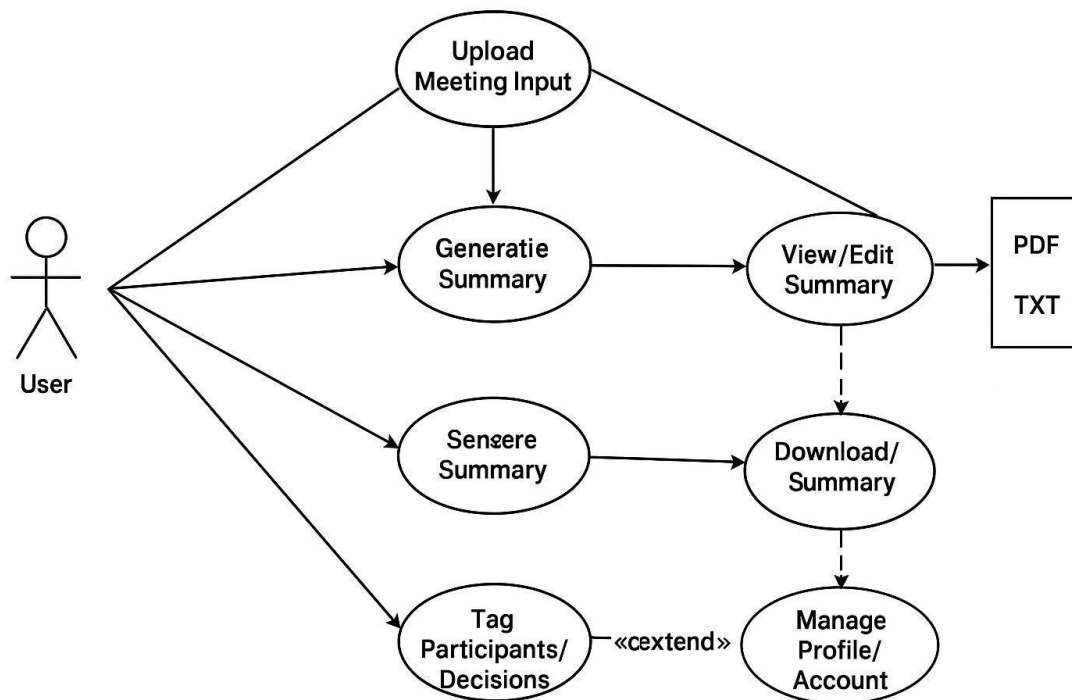
2.5 User Classes and Characteristics

User Class	Description
Admin/Owner	Manages platform, monitors API usage, and configures models
Corporate Users	Uploads meetings and views summaries
Educators/Students	Use it for class discussions and review
Freelancers	Keeps track of client conversations

2.6 Design and Implementation Constraints

- **Hardware Constraints:**
 - Minimum 8GB RAM for local processing
 - GPU preferred for model inference
 - 2-core CPU minimum
- **Software Constraints:**
 - Python 3.8+
 - TensorFlow or PyTorch
 - Google Speech-to-Text API or OpenAI Whisper
- **Interfaces:**
 - Google Meet / Zoom (optional API integration)
 - Frontend: Gradio
 - Backend: Python
- **Security Considerations:**
 - Authentication for file uploads
 - Encryption for meeting data
 - GDPR-compliant data handling (if deployed)
- **Standards:**
 - Follows RESTful API design
 - JSON for data interchange
 - Follows PEP8 for Python coding standards

2.7 Design Diagrams



2.8 Assumptions and Dependencies

- **Assumptions:**

- User will provide clear audio or clean transcripts.
- Internet connectivity is available for API-based services.
- Users have basic technical knowledge to use a web-based tool.

- **Dependencies:**
 - Speech-to-text accuracy relies on external APIs (Google, Whisper).
 - Summarization quality depends on pre-trained transformer models.
 - Data privacy compliance depends on external storage providers (e.g., AWS/GCP).
 - System performance depends on hardware or cloud configuration

3 System Requirements

User Interface

The Meeting Summarizer requires a user-friendly UI for the following modules:

- **Login/Signup Page:** Secure user authentication interface.
- **Dashboard:** Upload meeting files, view summaries, access recent activity.
- **Summary Viewer/Editor:** Interface to view, highlight, and edit generated summaries.
- **Export Panel:** UI for choosing export formats (PDF, TXT).
- **Search and Tagging Interface:** Tag participants, decisions, and search summaries.
- **User Profile Section:** Update personal details and manage account.

3.2 Software Interface

- **Modular Design:**
 - Frontend communicates with backend via **REST APIs**.
 - Backend interacts with the NLP engine, database, and cloud storage.
- **Services:**
 - File upload (multipart/form-data)
 - Transcription service (audio to text)
 - Summarization service (using NLP models)
 - Data persistence and retrieval
- **API Communication:**
 - JSON-based request/response format
 - API endpoints for login, upload, summarize, fetch summaries, search, and export

3.3 Database Interface

- Database Used: PostgreSQL or MongoDB (based on design preference)
- Functionality:
 - Stores user details, uploaded files, transcriptions, summaries, and tags
- Operations:
 - Create, Read, Update, Delete (CRUD) operations
 - Indexed search for summaries
 - Relational (PostgreSQL) or Document-based (MongoDB) structure

3.4 Protocols

- Communication Protocol: HTTPS over REST for secure communication
- Security Protocols:
 - OAuth 2.0 for authentication
 - JWT tokens for session handling
- Data Transfer:
 - Upload/download over SSL encryption
 - Rate-limiting APIs to avoid misuse

- Synchronization:
 - Backend ensures data consistency across upload, summarize, and export modules

4 Non-Functional Requirements

4.1 Performance Requirements

- Response Time:
 - Summary generation within 10 seconds for text-based input
 - Audio transcription within 20–30 seconds for 5-minute clips
- Scalability:
 - Handle at least 100 concurrent users
- Availability:
 - 99.9% uptime with proper load balancing

4.2 Security Requirements

- Authentication:
 - Secure login/signup using hashed passwords
 - Optional 2-Factor Authentication
- Authorization:
 - Role-based access (user, admin)
- Data Security:
 - AES encryption for storing sensitive files
 - GDPR-compliant user data handling
- Validation:
 - Input validation for uploads and text entries
 - Session expiry for inactive users

4.3 Software Quality Attributes

Attribute	Description
Adaptability	Easily integrable with other enterprise tools like Zoom or Teams.
Availability	24/7 availability with server auto-restart on failure.
Correctness	NLP engine trained to maintain accuracy of key information in summary.
Flexibility	Supports multiple input types – text, audio, video.
Interoperability	REST APIs can interact with third-party scheduling/meeting apps.
Maintainability	Modular code allows easy updates or patching.
Portability	Deployable on cloud or on-premise systems.
Reliability	Robust exception handling and testing ensures minimum failure.
Reusability	Summarization engine can be reused for other document types.
Robustness	Handles corrupted files and unexpected inputs gracefully.
Testability	Unit and integration tests automated via CI/CD pipelines.
Usability	Simple, clean UI with helpful tooltips and support docs.

4.4 OTHER REQUIREMENTS

- Language Support: Extendable for multilingual meetings in future versions.
- Analytics Module (optional): Track time saved per summary, keyword frequency.
- Cloud Support: AWS S3 or Firebase for file storage, scalable compute services for transcription/summarization.
- Mobile Compatibility: Responsive design or PWA (Progressive Web App).

5 Project Plan

The following Gantt chart represents the projected timeline and development milestones for the proposed

Meeting Summarizer project, outlining the sequential progression of activities from conceptualization to final deployment. The project commenced at the **end of August 2025** and is structured across several key development phases over the following weeks.

Timeline – Starting Late August 2025		
Phase	Activities	Duration
Week 1–2 (Late Aug – Early Sept)	Ideation & Requirement Analysis – Defining project objectives, understanding user needs, identifying core technologies (Whisper.cpp, Ollama, Gradio, FFmpeg).	2 weeks
Week 3–4 (Mid Sept)	System Design & Proposal Development – Preparing detailed design documentation, architecture diagrams, and project proposal submission.	2 weeks
Week 5–6 (Late Sept – Early Oct)	Audio Processing Module Development – Implementing FFmpeg-based preprocessing and ensuring compatibility with whisper.cpp.	2 weeks
Week 7–8 (Mid Oct)	Speech-to-Text (Whisper.cpp) Integration – Building and integrating whisper.cpp for efficient local transcription.	2 weeks
Week 9–10 (Late Oct)	Text Summarization Module (Ollama Integration) – Connecting to local Ollama server, testing LLM-based summarization, and optimizing prompts.	2 weeks
Week 11–12 (Early Nov)	Web UI Development with Gradio – Designing and implementing the user interface for audio upload, context input, and model selection.	2 weeks
Week 13–14 (Mid Nov)	System Integration & Workflow Automation – Linking all modules into a unified pipeline, adding automation scripts and model selection logic.	2 weeks
Week 15–16 (Late Nov – Early Dec)	Testing & Optimization – Conducting functionality, accuracy, and performance tests; optimizing model parameters and system speed.	2 weeks
Week 17–18 (Mid Dec)	Documentation & Reporting – Preparing user manuals, technical documentation, and performance evaluation reports.	2 weeks
Week 19 (Late Dec)	Final Presentation & Deployment – Presenting the completed project, demonstrating end-to-end functionality, and deploying the local Gradio interface.	1 week

References

Sadia, B., Adeeba, F., Shams, S., & Javed, K. (2024). Meeting the challenge: A benchmark corpus for automated Urdu meeting summarization. *Information Processing & Management*, 61(4),103734.

Brahmachari, K. S., Hanumesh, M., Anitha, G., & Pattanaik, B. (2024, June). Efficient Meeting Insights: NLP-Enhanced Summarization of Voice and Text. In *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)* (pp.1-8).IEEE.

Vadlamudi, G., Vemuru, N., Vangapalli, S., Surapaneni, R. K., & Nimmagadda, S. (2022, April). Meeting summarizer using Natural Language Processing. In *2022 6th International Conference on Trends in Electronics and Informatics (ICOEI)* (pp. 1610-1614).IEEE.

Conclusion

Summary

An AI-Powered Meeting Summarizer that automatically extracts important insights, action items, and succinct summaries from recorded meetings was successfully created as part of this project. The solution uses a Large Language Model (LLM) via Ollama to provide logical and context-aware summaries and incorporates cutting-edge speech-to-text technology using Whisper.cpp for precise transcription, allowing for effective comprehension of meeting outcomes.

The technology facilitates greater cooperation by providing concise, organized summaries and increases productivity by cutting down on the amount of time needed to evaluate lengthy talks. Transcription, natural language processing, and user interaction via a web-based interface were handled by core technologies and libraries like Python, Whisper.cpp, Ollama, and Gradio.

Limitations

- **AI Model Accuracy:** The accuracy of the underlying speech-to-text model (Whisper.cpp) and the LLM employed via Ollama determine the quality of transcriptions and summaries. Accents, background noise, overlapping speech, and audio quality can all have an impact on output accuracy.
- **Context Preservation:** Extended discussions or extremely complicated interactions may have poor context retention due to LLM context window limits, thus compromising summary completeness, even while summaries catch important information.
- **Backend Dependency:** The Ollama service must be operational and available for the system to function. System availability and summary generation may be impacted by summary generation and system availability.

•